

Java Backend Architecture

**Interview Series**

# Chapter 17.6

## Autoscaling

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

## Chapter 17.6 — Autoscaling

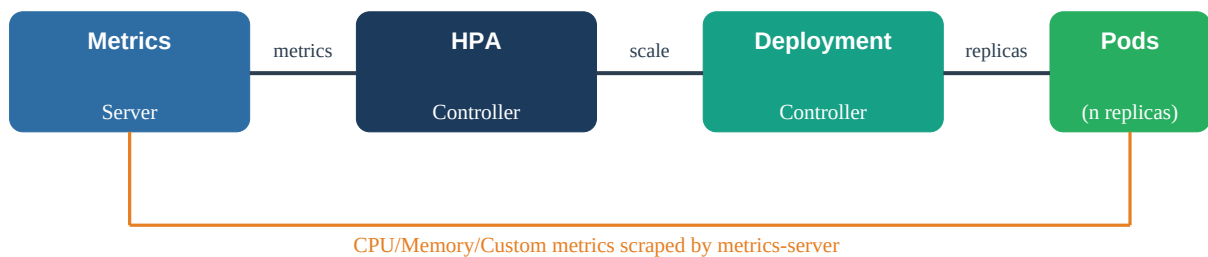
### Overview

Kubernetes provides three autoscaling dimensions: HPA (pod replicas), VPA (pod resource requests), and Cluster Autoscaler (node count). KEDA extends HPA with event-driven scaling including scale-to-zero capability.

### Horizontal Pod Autoscaler (HPA)

#### HPA Feedback Control Loop (15s scrape interval)

$$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{desiredMetric})$$



- **Metrics Server** required for CPU/memory HPA (lightweight metrics aggregator)
- **HPA v2** supports multiple metrics: resource (cpu/memory), pods, object, external
- **averageUtilization: 70** — scale when avg CPU > 70% of requested
- **stabilizationWindowSeconds** — prevent thrashing: scale-down stabilization 300s default
- **scaleDown.policies** — limit scale-down rate: e.g., max 10% per 60s

```
apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: api-hpa spec:
  scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: api minReplicas: 2 maxReplicas: 20
  metrics:
    - type: Resource resource: name: cpu target: type: Utilization averageUtilization: 70
    - type: Resource resource: name: memory target: type: Utilization averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
    policies:
      - type: Percent value: 10
      periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
    policies:
      - type: Percent value: 100
      periodSeconds: 30
```

### KEDA — Kubernetes Event-Driven Autoscaling

### KEDA: Kafka Consumer Lag Scaler (scale-to-zero capable)



$\text{desiredReplicas} = \text{ceil}(\text{lag} / \text{lagThreshold}) = \text{ceil}(5000/100) = 50 \text{ replicas}$

When lag = 0: scale to 0 replicas (ScaledObject minReplicaCount: 0)

- **Scale-to-zero:** minReplicaCount: 0 — pods disappear when idle
- **60+ scalers:** Kafka, SQS, RabbitMQ, Azure Service Bus, Prometheus, MySQL, Redis, Cron...
- **ScaledObject:** scales Deployments/StatefulSets based on external triggers
- **ScaledJob:** creates Job per event from queue (parallel processing)
- KEDA creates and manages an HPA — visible via `kubectl get hpa`

```

apiVersion: keda.sh/v1alpha1 kind: ScaledObject metadata: name: kafka-consumer-scaler spec:
scaleTargetRef: name: kafka-consumer minReplicaCount: 0 # scale to zero! maxReplicaCount: 50
pollingInterval: 15 cooldownPeriod: 300 triggers: - type: kafka metadata: bootstrapServers:
kafka:9092 consumerGroup: order-processors topic: orders lagThreshold: "100" # 1 replica per
100 messages lag offsetResetPolicy: latest --- # SQS scaler triggers: - type: aws-sqs-queue
metadata: queueURL: https://sqs.us-east-1.amazonaws.com/123/orders queueLength: "10"
awsRegion: us-east-1 authenticationRef: name: aws-trigger-auth # TriggerAuthentication with
IRSA
  
```

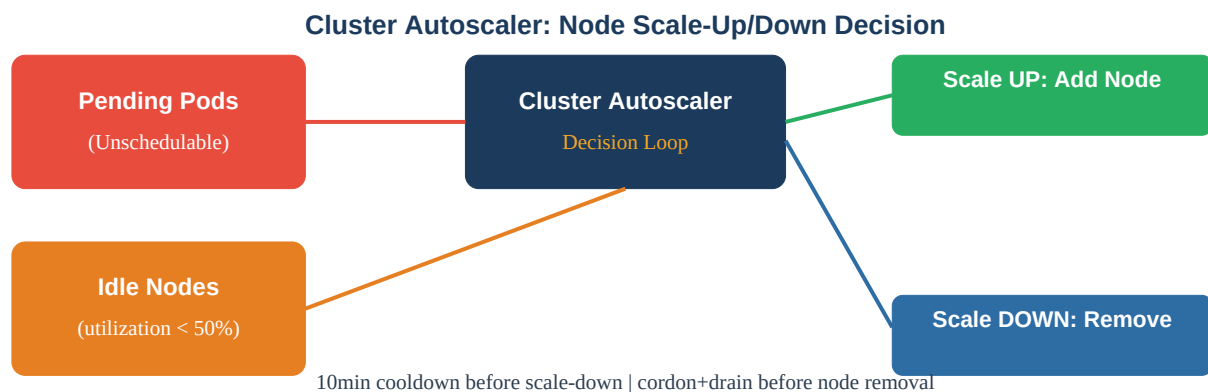
## Vertical Pod Autoscaler (VPA)

- **Recommender:** analyzes historical CPU/memory usage, recommends request/limit values
- **Updater:** evicts pods to apply new resource recommendations (causes restarts)
- **Admission Plugin:** mutates pod spec at creation with recommended values
- **updateMode: Off** — recommendations only, no action (safe start)
- **updateMode: Initial** — apply on pod creation only, no eviction
- **updateMode: Auto** — evict and recreate pods (production risk: avoid for stateful)
- VPA and HPA conflict on CPU/memory — use HPA on custom metrics + VPA for right-sizing

```

apiVersion: autoscaling.k8s.io/v1 kind: VerticalPodAutoscaler metadata: name: api-vpa spec:
targetRef: apiVersion: apps/v1 kind: Deployment name: api updatePolicy: updateMode: "Off" #
Recommend only resourcePolicy: containerPolicies: - containerName: api minAllowed: cpu: 100m
memory: 128Mi maxAllowed: cpu: 4 memory: 4Gi controlledResources: ["cpu", "memory"] # Check
recommendations: # kubectl describe vpa api-vpa #
Status.Recommendation.ContainerRecommendations[].Target
  
```

## Cluster Autoscaler



- **Scale-up trigger:** pod in Pending due to insufficient resources — CA finds node group that can fit it
- **Scale-down trigger:** node utilization < 50% AND all pods can be rescheduled elsewhere
- **10 minute cooldown** before considering scale-down; 3-minute cooldown for scale-up
- **PodDisruptionBudget:** CA respects PDB when draining nodes for scale-down
- **safe-to-evict: false** annotation prevents CA from evicting critical pods
- Works with node groups: AWS ASG, GCP MIG, Azure VMSS

```
# AWS EKS: Cluster Autoscaler deployment (after IAM policy setup) kubectl apply -f
cluster-autoscaler.yaml # Key args: # --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,k8s.io/cluster-autoscaler/my-cluster # --balance-similar-node-groups=true #
--skip-nodes-with-system-pods=false # --expander=least-waste (binpacking strategy) # Annotate
node to prevent scale-down kubectl annotate node node1
cluster-autoscaler.kubernetes.io/scale-down-disabled=true # Prevent pod from being evicted
during scale-down annotations: cluster-autoscaler.kubernetes.io/safe-to-evict: "false"
```

## Resource Requests vs Limits

| Field           | Purpose              | Scheduler Uses    | Overcommit Allowed         |
|-----------------|----------------------|-------------------|----------------------------|
| requests.cpu    | Guaranteed CPU share | Yes — bin packing | Yes — capped by limit      |
| limits.cpu      | Max CPU (throttled)  | No                | Yes — CPU throttled        |
| requests.memory | Guaranteed memory    | Yes               | No — OOMKilled if exceeded |
| limits.memory   | Max memory           | No                | No — OOMKill trigger       |

## PodDisruptionBudget (PDB)

- PDB limits voluntary disruptions (CA scale-down, node drain, rolling update)
- **minAvailable: 2** — at least 2 pods always running
- **maxUnavailable: 1** — at most 1 pod can be down
- CA will NOT drain a node if it would violate PDB

```
apiVersion: policy/v1 kind: PodDisruptionBudget metadata: name: api-pdb spec: minAvailable: 2 #
or: maxUnavailable: 1 selector: matchLabels: app: api
```

## Interview Q&A; — Autoscaling

### Q1. What is the difference between HPA, VPA, and Cluster Autoscaler?

HPA (Horizontal Pod Autoscaler) scales the number of pod replicas based on metrics. VPA (Vertical Pod Autoscaler) adjusts the CPU/memory requests of individual pods. Cluster Autoscaler adjusts the number of nodes in the cluster. They operate at different levels: HPA=replicas, VPA=pod resources, CA=nodes. Typically used together: HPA for dynamic load, VPA for right-sizing, CA for node elasticity.

### Q2. How does KEDA differ from HPA?

HPA scales based on Kubernetes-native metrics (CPU, memory, custom metrics via Metrics API). KEDA extends this with 60+ event source scalers (Kafka, SQS, RabbitMQ, Prometheus, etc.) and enables scale-to-zero (minReplicaCount: 0). KEDA creates and manages an HPA under the hood. HPA alone cannot scale to zero. KEDA's ScaledJob creates one Job per event from queues.

### Q3. How does HPA calculate the desired replica count?

$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} * \text{currentMetricValue} / \text{desiredMetricValue})$ . Example: currentReplicas=5, currentCPU=85%, targetCPU=70% ->  $\text{ceil}(5 * 85/70) = \text{ceil}(6.07) = 7$ . HPA fetches metrics from the Metrics API every 15 seconds. It applies stabilization windows to prevent flapping: default scaleDown.stabilizationWindowSeconds is 300s (keeps max replica count seen in window).

### Q4. What is the HPA stabilizationWindowSeconds and why is it important?

StabilizationWindowSeconds prevents rapid replica count oscillation. For scale-down, HPA looks back over stabilizationWindowSeconds (default 300s) and uses the MAXIMUM desired replica count seen in that window. This means if CPU spikes then drops, HPA waits 5 minutes before scaling down. For scale-up, stabilizationWindowSeconds is 0 by default for fast response. Tune scaleDown.stabilizationWindowSeconds based on your traffic patterns.

### Q5. What metrics can HPA v2 use?

Resource metrics: cpu, memory (as averageUtilization or averageValue). Pods metrics: custom metrics per pod (e.g., http\_requests\_per\_second). Object metrics: metrics from a specific k8s object (e.g., Ingress RPS). External metrics: metrics from outside k8s (e.g., SQS queue depth via Prometheus adapter). Container metrics (v2): resource usage of specific container within pod.

### Q6. What is the metrics pipeline for custom HPA metrics?

Flow: Application exposes Prometheus metrics -> Prometheus scrapes -> Prometheus Adapter translates to Kubernetes custom.metrics.k8s.io API -> HPA queries adapter. Alternatively: KEDA ScaledObject directly queries external metric sources without needing Prometheus Adapter. The metrics-server handles only built-in resource metrics (CPU/memory).

### Q7. Why can't you use HPA and VPA together on CPU/memory?

HPA scales replicas based on CPU utilization relative to requests. If VPA changes the requests value, it changes the utilization calculation, causing HPA to scale differently. This creates a feedback loop where VPA and HPA fight each other. Solution: use VPA with updateMode: Off for recommendations, then set requests manually. Or use HPA with custom/external metrics (not CPU/memory) and VPA for right-sizing.

### Q8. How does KEDA scale-to-zero work?

When minReplicaCount: 0 and the metric value drops to zero (e.g., Kafka lag = 0), KEDA bypasses HPA (which can't go below 1) and directly scales the deployment to 0 replicas. When a new event arrives, KEDA's ScaledObject detects it and scales to at least 1 replica, then HPA takes over for further scaling. There's an activation threshold to avoid scaling for a single message: activationLagThreshold configures this.

**Q9. What triggers Cluster Autoscaler scale-up?**

CA watches for pods in Pending state with reason Unschedulable. It simulates scheduling on each available node group and selects the cheapest/least-waste group that can fit the pending pods. It then triggers the cloud provider API (AWS ASG, GCP MIG) to add a node. CA does NOT proactively add nodes — it only reacts to pending pods.

**Q10. What prevents Cluster Autoscaler from scaling down a node?**

1) System-critical pods (kube-system) that cannot be moved. 2) Pods with PodDisruptionBudget that would be violated. 3) Pods without a controller (standalone pods — not managed by Deployment/RS). 4) Pods with safe-to-evict: false annotation. 5) Pods using local storage (hostPath or emptyDir with data). 6) Node utilization above 50% threshold (configurable).

**Q11. What is the difference between resource requests and limits?**

Requests: minimum guaranteed resources — used by scheduler for bin-packing decisions. A pod is scheduled only if a node has enough free requested capacity. Limits: maximum resources allowed — CPU is throttled if exceeded, memory causes OOMKill. With no limits set, pod can use all available node resources (noisy neighbor problem). Best practice: set requests = expected average, limits = expected peak (not too tight).

**Q12. What happens when a pod exceeds its memory limit?**

The container is OOMKilled (Out Of Memory Killed) immediately — no graceful shutdown. kubelet then restarts the container per the pod's restartPolicy (default Always). Repeated OOMKills increase the CrashLoopBackOff delay. Monitor: kubectl describe pod shows OOMKilled in last state. Fix: increase memory limit, profile for memory leaks, or use VPA recommendations.

**Q13. What is CPU throttling and how do you detect it?**

When a container's CPU usage hits its CPU limit, the Linux CFS scheduler throttles it — the container is paused for some CPU cycles. This causes latency spikes without OOMKill or restart. Detection: container\_cpu\_throttled\_seconds\_total and container\_cpu\_cfs\_throttled\_periods\_total Prometheus metrics. Fix: increase CPU limit, or remove limit with only request set (risky on shared nodes).

**Q14. How does PodDisruptionBudget interact with rolling updates?**

During a rolling update, the Deployment controller evicts old pods and creates new ones. PDB enforces minAvailable/maxUnavailable. If minAvailable: 3 and you have 3 replicas, the rolling update cannot remove any pod until a new one is Ready — effectively making updates serial. Set maxUnavailable in both PDB and Deployment.spec.strategy.rollingUpdate.maxUnavailable consistently.

**Q15. What is the KEDA TriggerAuthentication resource?**

TriggerAuthentication (or ClusterTriggerAuthentication) provides credentials for KEDA scalers. Instead of hardcoding credentials in ScaledObject, reference a TriggerAuthentication that reads from a k8s Secret or uses pod identity (IRSA, Workload Identity). For AWS: IRSA annotation on KEDA operator SA grants access to SQS/CloudWatch without secrets.

**Q16. How do you debug HPA not scaling?**

kubectl describe hpa — check Events and Conditions. Common issues: 1) metrics-server not running or returning errors. 2) Pod lacks resource requests (HPA needs requests to calculate utilization). 3) Already at maxReplicas. 4) Stabilization window preventing scale-down. 5) Metrics returning unknown — check: kubectl get --raw '/apis/metrics.k8s.io/v1beta1/pods'. 6) HPA disabled: check kube-controller-manager logs.

**Q17. What is the node expander strategy in Cluster Autoscaler?**

Expander determines which node group to scale when multiple groups can fit pending pods. random: choose randomly (default, fast). least-waste: choose group that wastes fewest resources (bin-packing, cost-efficient). most-pods: choose group that can schedule most pending pods. priority: user-defined priority per node group. least-waste is recommended for cost optimization.

#### Q18. How do you limit CA scale-down during business hours?

KEDA supports Cron scalers for time-based scaling. For CA, pre-scale using HPA with a Prometheus metric that returns minimum pod count based on time. Alternatively, set minSize on the node group (CA respects ASG min). Or use a CronJob that sets HPA minReplicas before business hours. CA expander priority can deprioritize scale-down groups during critical windows.

#### Q19. What is the ScaledJob in KEDA and when would you use it?

ScaledJob creates a new Kubernetes Job for each item in a queue instead of maintaining long-running consumer pods. Ideal for: batch image processing, ML inference per item, one-shot data transformation. Each job processes one message and exits. Scale behavior: multiplier controls jobs per queue item. Contrast with ScaledObject which scales persistent Deployment replicas.

#### Q20. How does VPA right-sizing work in practice?

1) Deploy VPA with updateMode: Off. 2) Run production workload for 7-14 days to collect data. 3) kubectl describe vpa shows Target, LowerBound, UpperBound recommendations. 4) Set pod requests to Target recommendation. 5) Optionally enable updateMode: Initial for new pods to get updated requests. Result: pods right-sized, scheduler makes better bin-packing decisions, cluster utilization improves, CA makes fewer unnecessary scale-ups.

---

#### Q21. How do you set up KEDA with Prometheus metrics?

Use the prometheus scaler. KEDA queries your Prometheus endpoint for a PromQL expression. Example: scale based on `http_requests_per_second > threshold`. ScaledObject trigger: `type: prometheus, serverAddress: http://prometheus:9090, metricName: http_rps, query: sum(rate(http_requests_total[2m])), threshold: '100'`. KEDA divides current query value by threshold to determine desired replica count.

#### Q22. What is the activation phase in KEDA scale-from-zero?

When scaled to zero and a new event arrives, there's a cold-start latency — the pod must be scheduled, pulled, and started before processing the first event. `activationThreshold`: configures the minimum metric value to trigger activation. For latency-sensitive workloads, use `minReplicaCount: 1` to avoid zero. HTTP scaling with KEDA HTTP Add-on uses a queue proxy to buffer requests during cold start.

#### Q23. How does CA handle spot/preemptible instance termination?

Spot instances can be terminated by the cloud provider with 2-minute notice. Node Termination Handler (AWS) or similar daemonset watches for termination notices, cordons the node, and triggers graceful pod draining. CA works alongside this — when spot nodes drain, pending pods trigger CA to provision new nodes. Mix spot + on-demand node groups; use priority expander to prefer spot.

#### Q24. What is Goldilocks and how does it help with right-sizing?

Goldilocks is a VPA-based dashboard tool that deploys VPA in Off mode for all workloads and provides a web UI showing recommendations. It helps teams visualize over/under provisioned deployments across the cluster. Combined with `kubectl-resource-capacity` plugin, it enables data-driven right-sizing without manual VPA interpretation.

#### Q25. How do you handle HPA during deployments with many new pod starts?

During a rolling update, new pods start and temporarily show low CPU (before load hits them), which may trigger HPA to scale down prematurely. Mitigation: 1) Set `scaleDown.stabilizationWindowSeconds` to cover

deployment duration. 2) Use readiness probes — HPA ignores unready pods in metric calculation. 3) Set minReplicas to cover expected load during deployment. 4) Use HPA behavior with conservative scaleDown policies.

**Q26. What is the difference between cluster-autoscaler and Karpenter?**

Cluster Autoscaler works with pre-defined node groups (ASG, MIG). It selects which group to scale based on pending pods. Karpenter (AWS) directly provisions EC2 instances optimized for pending pod requirements — no pre-defined node groups. Karpenter is faster (sub-60s vs CA's 3-4 minutes), bins packs better, and supports consolidation (replacing multiple small nodes with one large node). Karpenter is preferred for AWS; CA remains common on GKE/AKS.